SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Name: K VENKATA VAMSI GOUD	Reg. No.: 192472021
Course Code: DSA0304	Course Name: Natural Language Processing
CO Assessed: CO2 – Manipulation	Assessment: AT3 – Error Analysis

Question 1 – Morphological Analysis Error in a Biomedical Dataset

Aim: Examine how Porter stemming affects infection, infectious, infected and infect, and explain why stemming alone can reduce biomedical search accuracy.

1. Preprocessing and observed stems

- Load the biomedical corpus and select the text field.
- Convert text to lowercase and tokenize it into individual words; remove punctuation where appropriate.
- Apply PorterStemmer to each token and compare the outputs with the original words.

Original word	Porter stem (NLTK)	Analysis
infection	infect	Suffix -ion is removed; the relationship to the verb infect remains recognizable.
infectious	infecti	The result is an artificial-looking stem and obscures the adjective suffix -ious.
infected	infect	The inflectional past-tense suffix -ed is removed.
infect	infect	Already the base form; unchanged.

2. Error and correction

The main problem is applying a general-purpose stemmer before morphological analysis. Porter stemming is rule-based and does not preserve every derivational distinction. In particular, infectious becomes infecti, which is not a normal dictionary lemma. This can make biomedical terms difficult to interpret and may group words too broadly or inconsistently.

Inflectional morphology usually changes grammatical form (infected → infect) while retaining the same basic word meaning. Derivational morphology can create a new word class or meaning (infect → infectious). For clinical search, preserve the original term and use lemmatization or a biomedical vocabulary-aware normalizer alongside stemming.

3. Example Python experiment

```
import pandas as pd
import re
from nltk.stem import PorterStemmer

ps = PorterStemmer()
data = pd.read_csv("PubMed20k.csv")  # change to the actual dataset filename
text_col = "text"                    # change to the actual text-column name
words = ["infection", "infectious", "infected", "infect"]

for word in words:
    print(word, "->", ps.stem(word))

# Basic corpus preprocessing example
data["tokens"] = data[text_col].fillna("").str.lower().apply(
    lambda s: re.findall(r"\\b[a-z]+\\b", s)
)
data["stems"] = data["tokens"].apply(lambda ts: [ps.stem(w) for w in ts])
print(data[["tokens", "stems"]].head())
```

Conclusion: Use stemming as a retrieval aid, not as the only representation. Keep original terms and evaluate retrieval precision/recall on the chosen corpus.

Question 2 – Error Analysis of a Finite-State Parser in Social Media Data

Problem: The parser fails when a word contains more than one affix. A finite-state transducer (FST) should allow valid sequences of prefix, root and suffix transitions instead of checking only one affix.

Word	Correct analysis	Type / note
happiest	happy + -est (spelling change y → i)	Inflectional superlative
unbelievable	un- + believe + -able	Derivational prefix and suffix
running	run + -ing (double n in spelling)	Inflectional; spelling rule applies
reordering	re- + order + -ing	Prefix plus inflectional suffix
smartphones	smartphone + -s	Plural inflection
unreadable	un- + read + -able	Derivational prefix and suffix

Why it fails and how to modify it

A single-affix parser has too few transitions and may stop after recognizing a prefix or suffix. It may also lack spelling rules such as y → i (happy → happiest) and consonant doubling (run → running). Add states that permit a prefix, a root, and one or more suffixes, while checking the sequence against a lexicon and spelling rules.

```
# Simplified FST-style rule sequence (illustrative)
def analyze(word):
    prefixes = ["un", "re"]
    suffixes = ["able", "ible", "est", "ing", "s"]
    original = word
    prefix = ""
    suffix = ""

    for p in prefixes:
        if word.startswith(p) and len(word) > len(p) + 2:
            prefix, word = p, word[len(p):]
            break

    # Handle common spelling changes before suffix analysis
    if word == "happiest":
        return original, "happy", "happy + est (y -> i)"
    if word == "running":
        return original, "run", "run + ing (consonant doubling)"
    if word.endswith("ing"):
        suffix, word = "ing", word[:-3]
        if word.endswith(word[-1:]) and len(word) > 1:
            # production systems should validate doubling with a lexicon/rule
            pass
    elif word.endswith("est"):
        suffix, word = "est", word[:-3] + "y" if word.endswith("i") else word[:-3]
    elif word.endswith("able"):
        suffix, word = "able", word[:-4]
    elif word.endswith("s"):
        suffix, word = "s", word[:-1]

    return original, word, f"{prefix} + {word} + {suffix}"

words = ["happiest", "unbelievable", "running",
         "reordering", "smartphones", "unreadable"]
for w in words:
    print(analyze(w))
```

Evaluation: Build a manually labelled test set, count a parse as correct only when its root and affixes match the gold analysis, and calculate accuracy = correct analyses / total words × 100. The assignment does not provide experimental counts, so numerical before/after accuracy should be measured rather than invented. A richer FST adds transitions and rule checks; with a fixed number of states, processing is generally linear in word length, although ambiguous paths can increase search cost.

Question 3 – Stemming Errors in News Article Classification

Stemming affects feature extraction by mapping related-looking words to a common feature. This can reduce vocabulary size, but over-stemming may merge words with different meanings and remove useful information for classifying technology and business news.

Word	Typical Porter output	Observation
organization	organ	Derivational suffixes are removed aggressively.
organizer	organ	Merged with organization.
organizing	organ	Verb form also maps to organ.
organized	organ	Past form maps to the same stem.
organization's	organization\'	Apostrophe handling can leave an awkward token if punctuation is not cleaned first.

Comparison and recommendation

- Without stemming: preserves word forms and semantics but creates a larger vocabulary.
- Porter stemming: reduces vocabulary and may improve matching, but can produce non-words and merge distinct terms.
- Lemmatization: uses vocabulary and grammatical information to return dictionary forms; it often preserves meaning better, though it requires more processing and suitable POS information.

Compare all three methods on the same train/test split and classifier. Report accuracy or macro-F1 and processing time; do not assume a numerical result without running the experiment. For minimizing semantic information loss, lemmatization is generally preferable. Keep original terms where domain-specific distinctions matter.

Question 4 – Morphological Errors in an E-commerce Search System

Porter stemming may improve matching between grammatical variants, but it can also combine words that have different roles or meanings. This can make product recommendations less relevant.

Word	Morphology	Likely Porter stem	Risk
watches	watch + -es; plural noun or verb form	watch	Usually useful inflectional normalization.
watching	watch + -ing	watch	Usually useful when it is a verb form; context can matter.
washable	wash + -able	washabl	Derivational suffix; “washable” describes a product property.
washer	wash + -er	washer	Agent/instrument noun; should not necessarily merge with wash.
washed	wash + -ed	wash	Inflectional past form; usually safe for verb search.

Improved strategy

- Normalize case and punctuation, but preserve the original query and product text.
- Use lemmatization with part-of-speech information for grammatical forms such as watches/watched/watching.
- Treat derivational suffixes such as -able and -er cautiously because they can express product attributes or different objects.
- Use domain-specific synonyms and product attributes, and test with relevance judgments, precision@k or nDCG.
- Apply stemming only as an additional recall mechanism, not as the sole matching rule.

Conclusion: Inflectional suffix removal is often helpful; derivational suffix removal requires semantic and product-domain checks.

Question 5 – Correcting a Porter Stemmer Program

Given code: data["Processed"] = data["Text"].apply(ps.stem). The error is that ps.stem expects one word, but each Text cell usually contains a complete sentence or article. The program therefore stems the whole string as if it were one token, instead of stemming each word. It may also fail if the CSV path or Text column name is incorrect or if values are missing.

Corrected program

```
import pandas as pd
import re
from nltk.stem import PorterStemmer

ps = PorterStemmer()
data = pd.read_csv("BBCNews.csv")

# Check that the expected column exists
if "Text" not in data.columns:
    raise ValueError("BBCNews.csv must contain a 'Text' column")

def stem_text(text):
    text = "" if pd.isna(text) else str(text)
    tokens = re.findall(r"\\b[a-zA-Z]+\\b", text.lower())
    return " ".join(ps.stem(token) for token in tokens)

data["Processed"] = data["Text"].apply(stem_text)
print(data[["Text", "Processed"]].head(10))
data[["Text", "Processed"]].to_csv("BBCNews_stemmed.csv", index=False)
```

Run `pip install pandas nltk` first. Place BBCNews.csv in the working directory or provide its full path. The dataset itself was not included with the assignment document, so the code is ready to run but dataset-specific output cannot be truthfully reported here.

Examples of 20 stemming cases

Original	Porter stem	Morphological note
connected	connect	Inflectional -ed
connecting	connect	Inflectional -ing
connection	connect	Derivational -ion
connectivity	connect	Derivational -ity
infection	infect	Derivational -ion
infectious	infecti	Derivational -ious; stem is not a dictionary lemma
infected	infect	Inflectional -ed
studies	studi	Inflectional plural/verb form; Porter stem is not “study”
studied	studi	Inflectional -ed; Porter stem is not “study”
studying	studi	Inflectional -ing; Porter stem is not “study”
running	run	Inflectional -ing with consonant doubling
runners	runner	Derivational -er plus plural -s; does not fully reduce to run
runs	run	Inflectional -s
organization	organ	Derivational -ization; aggressive reduction
organizer	organ	Derivational -er; merges with organization
organizing	organ	Inflectional -ing after derivational formation; aggressive reduction
organized	organ	Inflectional -ed after derivational formation; aggressive reduction
watches	watch	Inflectional -es
washed	wash	Inflectional -ed
washable	washabl	Derivational -able; property meaning may be lost

Interpretation: Porter outputs are retrieval stems, not guaranteed dictionary words. Several examples show that derivational forms can be merged or left with awkward stems. A fair evaluation should compare original tokens and stems on actual BBC articles and manually review at least 20 cases from that dataset. The examples above are representative test cases, not claimed observations from the missing BBCNews.csv file.

Overall conclusion

Morphological preprocessing must distinguish grammatical inflection from word-forming derivation. Porter stemming is fast and useful for recall, but it can lose meaning. Finite-state rules, lemmatization, domain lexicons and evaluation on labelled examples can improve consistency and retrieval quality.